

2-1. 클래스

- class 키워드 선언 하고 new 키워드로 사용함.
- constructor 함수를 사용하여 멤버 속성을 정의함 (this 사용)
- 모든 속성은 공개 속성임
- 메소드는 function을 사용 하지 않는다
- 자바 스크립트의 Class도 Prototype 이다
- Class를 사용 하면 간단히 사용 할 수 있다

자바스크립트 클래스는 class 표현식 and class 선언 두 가지 방법을 제공하며 constructor에 매개변수를 담아 매개변수들을 통해 프로퍼티와 프로퍼티 메소드를 만듭니다.

{% hint style="info" %} 엄격 모드는 평범한 JavaScript 시멘틱스에 몇가지 변경이 일어나게 합니다.

참고 : https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Strict_mode

```
"use strict";
```

1. 기존에는 조용히 무시되던 에러들을 throwing합니다.
2. JavaScript 엔진의 최적화 작업을 어렵게 만드는 실수들을 바로잡습니다. 가끔씩 엄격 모드의 코드는 비-엄격 모드의 동일한 코드보다 더 빨리 작동하도록 만들어집니다.
3. 엄격 모드는 ECMAScript의 차기 버전들에서 정의 될 문법을 금지합니다. {% endhint %}

1. 클래스 선언

함수 선언과 클래스 선언의 중요한 차이점은 함수의 경우 정의하기 하기 전에 호출할 수 있지만, 클래스는 반드시 정의한 뒤에 사용할 수 있다는 점입니다

1-1. 클래스 선언

```
class 클래스명 {  
  constructor(매개변수1, 매개변수2, ...){  
    this.프로퍼티1 = 매개변수 값;  
    this.프로퍼티2 = 매개변수 값;  
    ...  
  }  
  메소드명(매개변수1, 매개변수2, ...){  
    수행할 문장  
    ...  
  }  
  ...  
}
```

```

class Customer {
  //constructor로 생성자
  constructor(name, age) {
    //this.은 프로퍼티의 부모를 말한다.
    this.name = name;
    this.age = age;

    // 익명함수로 생성할 수도 있고
    this.fnHelloMsg = function(){
      console.log(this.name + "님 나이는" + this.age );
    }
  }

  //기명함수로 생성할 수도있다.
  fnByeBye(){
    console.log(this.name + "님 안녕 ~~~~!" );
  }
}

// 객체 생성
const cust = new Customer("홍길동", 30);
console.log(cust);
/**
 * Customer { name: '홍길동',
 *           age: 30,
 *           fnHelloMsg: f }
 * [[Prototype]]: Object
 * constructor: class Customer
 * fnByeBye: f fnByeBye()
 * [[Prototype]]: Object
 *   constructor: f
 *   ....
 */

cust.fnHelloMsg(); // 홍길동님 나이는30
cust.fnByeBye();  // 홍길동님 안녕 ~~~~!

const cust01 = new Customer("김길동", 40);
cust01.fnHelloMsg(); // 김길동님 나이는40

console.log(cust === cust01); // false
console.log(cust === cust);   // true

```

1-2. 클래스 표현식

```

const 클래스명 = class {
  constructor(매개변수1, 매개변수2, ...){
    this.프로퍼티1 = 매개변수 값;
    this.프로퍼티2 = 매개변수 값;
    ...
  }
}

```

```

    }
    메소드명(매개변수1, 매개변수2, ...){
        수행할 문장
        ...
    }
    ...
}

```

```

let Customer = class {
  //constructor로 생성자
  constructor(name, age) {
    //this.은 프로퍼티의 부모를 말한다.
    this.name = name;
    this.age = age;

    // 익명함수로 생성할 수도 있고
    this.fnHelloMsg = function(){
      console.log(this.name + "님 나이는" + this.age );
    }
  }

  //기명함수로 생성할 수도있다.
  fnByeBye(){
    console.log(this.name + "님 안녕 ~~~~!" );
  }
}

// 객체 생성
const cust = new Customer("홍길동", 30);
console.log(cust);
/**
 * Customer { name: '홍길동',
 *             age: 30,
 *             fnHelloMsg: f }
 * [[Prototype]]: Object
 *   constructor: class Customer
 *   fnByeBye: f fnByeBye()
 *   [[Prototype]]: Object
 *     constructor: f
 *     ....
 */

cust.fnHelloMsg(); // 홍길동님 나이는30
cust.fnByeBye();  // 홍길동님 안녕 ~~~~!

const cust01 = new Customer("김길동", 40);
cust01.fnHelloMsg(); // 김길동님 나이는40

```

```
console.log(cust === cust01); // false
console.log(cust === cust);   // true
```

2. 클래스 특성

- 클래스의 본문(body)은 strict mode에서 실행됩니다.
- constructor는 하나만 존재 해야 합니다.
- 프로토타입 메서드 형식을 따릅니다. ([메서드 정의](#)도 참조해보세요.)

2-1. 정적 메서드

static 키워드를 사용하여 정적 메서드를 정의 합니다. 정적 메서드는 클래스의 인스턴스화([instantiating](#)) 없이 호출되며, 클래스의 인스턴스에서는 호출할 수 없습니다.

```
class Car {
  static title = '지동차';
  // 생성자
  constructor(name, company) {
    this.name = name;
    this.company = company;
  }

  static printCar() { // 지동차 은 undefined의 Car
    console.log(`${Car.title} 은 ${this.company}의 ${this.name}`);
  }

  static sPrintCar() { // 지동차
    console.log(`${Car.title}`);
  }

  printCar() { // 지동차 은 현대의 그랜저
    console.log(`${Car.title} 은 ${this.company}의 ${this.name}`);
  }
}

const hCar = new Car('그랜저', '현대');
Car.printCar();
Car.sPrintCar();
hCar.printCar();
console.log(Car.title); // 지동차
```

```
class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }

  static displayName = "Point";
  static distance(a, b) {
    const dx = a.x - b.x;
    const dy = a.y - b.y;
```

```

    return Math.hypot(dx, dy);
  }
}

const p1 = new Point(5, 5);
const p2 = new Point(10, 10);
p1.displayName; // undefined
p1.distance; // undefined
p2.displayName; // undefined
p2.distance; // undefined

console.log(Point.displayName); // "Point"
console.log(Point.distance(p1, p2)); // 7.0710678118654755

```

2-2. this 바인딩

메서드를 변수에 할당 한 다음 호출하는 것과 같이, 정적 메서드나 프로토타입 메서드가 **this** 값 없이 호출될 때, **this** 값은 메서드 안에서 **undefined**가 됩니다. 이 동작은 **"use strict"** 명령어 없이도 같은 방식으로 동작하는데, **class** 문법 안에 있는 코드는 항상 strict mode 로 실행되기 때문입니다

```

class Animal {
  speak() {
    return this;
  }
  static eat() {
    return this;
  }
}

let obj = new Animal();
obj.speak(); // the Animal object
let speak = obj.speak;
speak(); // undefined

Animal.eat(); // class Animal
let eat = Animal.eat;
eat(); // undefined

```

2-3. 인스턴스 속성

인스턴스 속성은 반드시 클래스 메서드 내에 정의되어야 합니다.

```

class Rectangle {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
}

```

정적 (클래스사이드) 속성과 프로토타입 데이터 속성은 반드시 클래스 선언부 바깥쪽에서 정의되어야 합니다.

```
Rectangle.staticWidth = 20;
Rectangle.prototype.prototypeWidth = 25;
```

2-4. Field 선언

{% hint style="info" %} **Warning:** public과 private 필드 선언은 JavaScript 표준화 위원회에 [실험적 기능 \(stage 3\) TC39](#) 로 제안되어있습니다. 현재 이를 지원하는 브라우저는 제한적인 상태입니다만, [Babel](#) 과 같은 build 시스템을 사용하면 이 기능을 사용해볼 수 있습니다. {% endhint %}

```
// public
class Rectangle {
  height = 0;
  width;
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
}

// private
class Rectangle {
  #height = 0;
  #width;
  constructor(height, width) {
    this.#height = height;
    this.#width = width;
  }
}
```

참고: [public class fields \(en-US\)](#), [private class fields](#)

2-5. 상속

extends 키워드는 `_클래스 선언_`이나 `_클래스 표현식_`에서 다른 클래스의 자식 클래스를 생성하기 위해 사용됩니다.

subclass에 constructor가 있다면, "this"를 사용하기 전에 가장 먼저 `super()`를 호출해야 합니다.

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a noise.`);
  }
}
```

```

}

class Dog extends Animal {
  constructor(name) {
    super(name); // super class 생성자를 호출하여 name 매개변수 전달
  }

  speak() {
    console.log(`${this.name} barks.`);
  }
}

let d = new Dog("Mitzie");
d.speak(); // Mitzie barks.

```

참고: [msdn 클래스](#)

2-6. super 상위 호출

super 키워드는 객체의 부모가 가지고 있는 메서드를 호출하기 위해 사용됩니다. 이는 프로토타입 기반 상속보다 좋은 점 중 하나입니다.

```

class Cat {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a noise.`);
  }
}

class Lion extends Cat {
  speak() {
    super.speak();
    console.log(`${this.name} roars.`);
  }
}

let l = new Lion("Fuzzy");
l.speak();
// Fuzzy makes a noise.
// Fuzzy roars.

```

2-7 set, get 주의 사항

```

class Car {
  // 생성자
  constructor(name, company) {

```

```

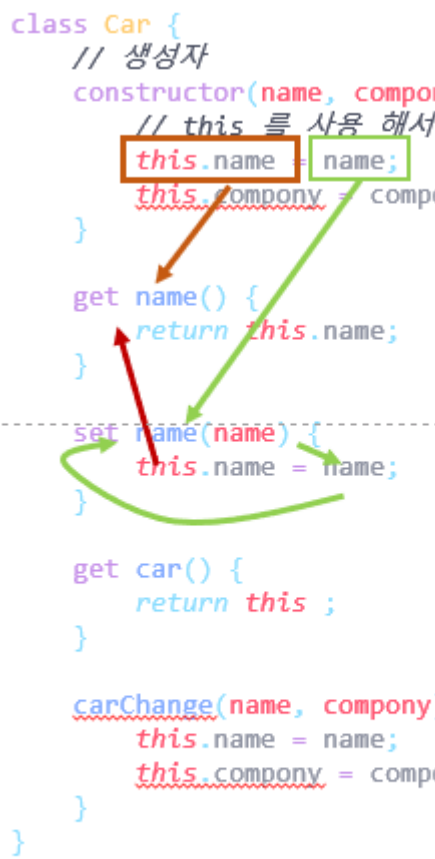
    // this 를 사용 해서 속성 설정
    this.name = name;
    this.compony = compony;
  }
  get name() {
    return this.name;
  }
  set name(name) {
    this.name = name;
  }

  get car() {
    return this ;
  }
  carChange(name, compony) {
    this.name = name;
    this.compony = compony;
  }
}

const car = new Car('그랜저', '현대');
console.log(car.name);

```

문제점 : 순환에 의해서 메모리 오류 발생



```

class Car {
  // 생성자
  constructor(name, compony) {
    // this 를 사용 해서 속성 설정
    this.name = name;
    this.compony = compony;
  }

  get name() {
    return this.name;
  }

  set name(name) {
    this.name = name;
  }

  get car() {
    return this ;
  }

  carChange(name, compony) {
    this.name = name;
    this.compony = compony;
  }
}

const car = new Car('그랜저', '현대');
console.log(car.name);

```

Uncaught RangeError: Maximum call variable.js:14
stack size exceeded

- at Car.set name [as name] (variable.js:14)
- at Car.set name [as name] (variable.js:14)
- at Car.set name [as name] (variable.js:14)
- at Car.set name [as name] (variable.js:14)
- at Car.set name [as name] (variable.js:14)
- at Car.set name [as name] (variable.js:14)
- at Car.set name [as name] (variable.js:14)
- at Car.set name [as name] (variable.js:14)
- at Car.set name [as name] (variable.js:14)
- at Car.set name [as name] (variable.js:14)

수정 : 속성으로 접근해야 함


```
class Car {  
  // 생성자  
  constructor(name, compony) {  
    // this 를 사용 해서 속성 설정  
    this.name = name;  
    this.compony = compony;  
  }  
  get name() {  
    return this._name;  
  }  
  set name(name) {  
    this._name = name;  
  }  
  
  get car() {  
    return this ;  
  }  
  carChange(name, compony) {  
    this.name = name;  
    this.compony = compony;  
  }  
}  
  
const car = new Car('그랜저', '현대');  
console.log(car.name); // 그랜저
```